

How a hawker gets an agent

What the catalog agent actually does to a messy spreadsheet, what sits behind the widget, and how someone who has only ever used WhatsApp ends up with a working commerce agent.

Part 1 · What the catalog agent does

WORKED EXAMPLE

A real stall's menu never arrives clean. Here is a deliberately typical one — inconsistent capitalisation, a stray "\$", a row that is a section header rather than a dish, the same item entered twice, and one price simply missing. On the right is the tree the agent produces from it.

WHAT THE MERCHANT SENDS			MENU.CSV
STALL	Menu Item	\$	remarks
sakura japanese	CHICKEN KATSU CURRY DON	\$6.80	
Sakura Japanese	Salmon Mentai Don	7.50	raw salmon
sakura japanese	Chicken Karaage (6pc)	5.00	
--- DRINKS ---			
Sakura Japanese	Green Tea (Iced)	1.80	no sugar option
Ah Huat Mixed Rice	ECONOMY RICE 2 VEG 1 MEAT	3.20	
Ah Huat Mixed Rice	Economy Rice - 2 veg 1 meat	3.20	
Ah Huat Mixed Rice	Braised Pork Belly w Rice	4.50	pork
Ah Huat Mixed Rice	Sambal Egg		add on

▼	Ridge View Canteen	CANTEEN	
└▼	Sakura Japanese	STALL	
└└▼	Donburi	AGENT	
└└└	Chicken Katsu Curry Don		6.80
└└└	Salmon Mentai Don		7.50
└└▼	Sides	AGENT	
└└└	Chicken Karaage (6pc)		5.00
└└▼	Drinks	AGENT	
└└└	Iced Green Tea		1.80
└└▼	Ah Huat Mixed Rice	STALL	
└└└▼	Economy rice	AGENT	
└└└└	Economy Rice — 2 Veg 1 Meat		3.20
└└└└	merged 2 duplicate rows		
└└└▼	Rice plates	AGENT	
└└└└	Braised Pork Belly with Rice		4.50
└└▼	Add-ons	AGENT	
└└└	Sambal Egg	NO PRICE	

× row 4 dropped — section header, not a dish

Five things happened there that a spreadsheet import cannot do: it **invented categories that fit this stall** (Donburi, not "Mains"), **normalised names** across inconsistent casing, **merged an exact duplicate**, **recognised a section header as junk** rather than importing it as a \$0 dish, and **flagged the one row it could not resolve** instead of guessing a price. The merchant confirms or corrects; nothing publishes on the agent's word alone.

And underneath every dish, the part customers actually search on

The tree is what the merchant sees. This is what gets stored — and it is why "something warm and soupy, not too heavy, no pork" is answerable at all.

```
{
  "name": "Chicken Katsu Curry Don", "stall": "Sakura Japanese", "price": 6.80,
  "description": "Crisp fried cutlet over rice under thick Japanese curry." ← written

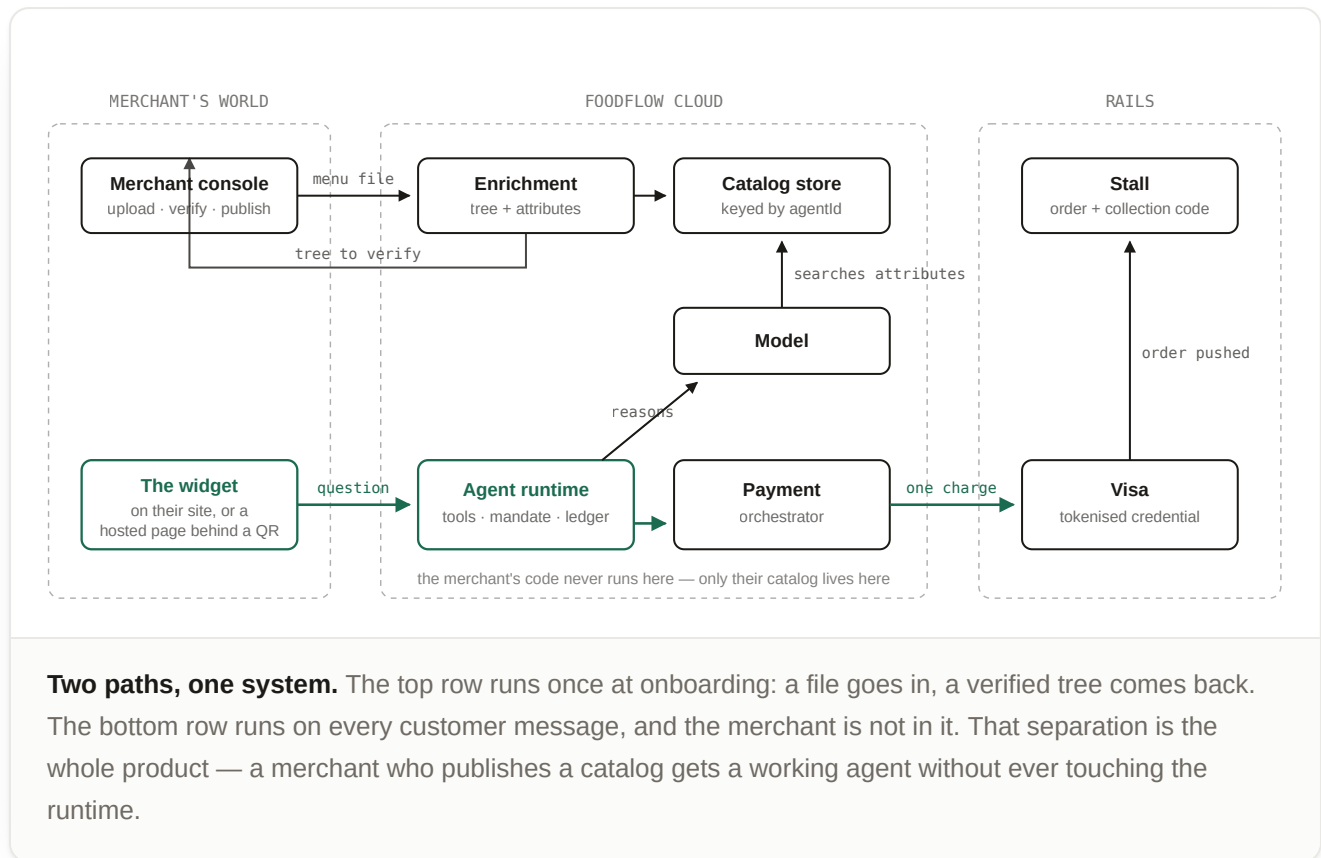
  "flavour": { "savoury": 3, "sweet": 2, "salty": 2, "spicy": 1, "sour": 0 },
  "texture": ["crispy", "saucy"], "cooking": "deep-fried", "temperature": "hot",
  "form": "rice", "heaviness": 5, "cuisine": "Japanes

  "dietary": [], // hard filters – never soft-scored, never guess
  "contains": ["chicken", "egg", "gluten"],
  "allergens": ["egg", "gluten", "soy"],

  "mealFit": ["lunch", "dinner"], "occasion": ["comfort"], "prepMinutes": 1

  "confidence": 0.86, "needsReview": false,
  "aiFields": ["description", "flavour", "texture", "allergens", "category"] ← drives
}
```

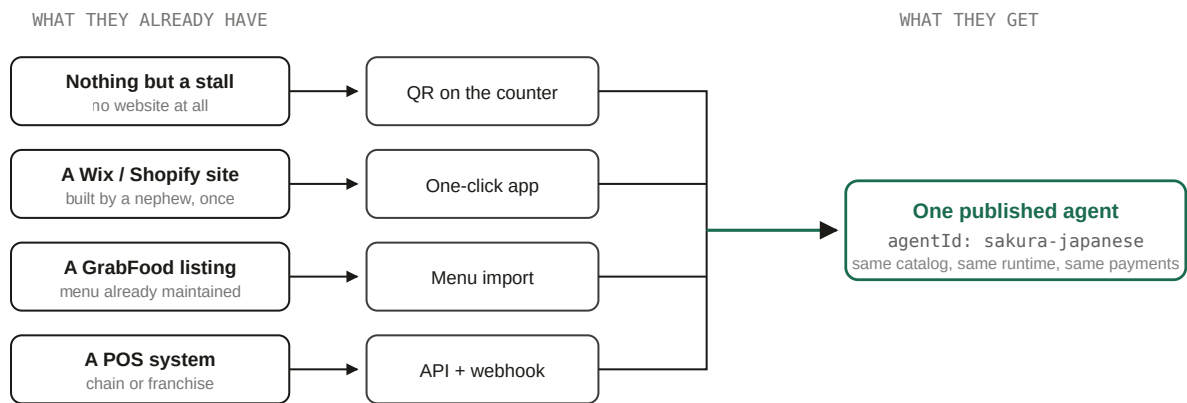
Your instinct about the two surfaces is right: the storefront is the consumer app, the console is where the thing gets configured. But the important part is what crosses the line between them — because the merchant never installs an agent. They publish a catalog, and FoodFlow serves the agent.



"How does a merchant add this to their website?" hides a bad assumption.

Almost no hawker stall has a website. Neither do most SMEs in this category. If the integration story is a `<script>` tag, the product is only for merchants who already have a developer — which is precisely the group that doesn't need us.

So the answer is not one integration. It is four doors, all opening onto the same agent. Which door a merchant walks through depends on what they already have, and none of the doors require them to write, paste, or understand a line of code.



Four doors, one runtime. The merchant's existing situation picks the door; behind all four is the identical agent. Adding a door is a distribution decision, not an architecture change — which is what makes the claim "any merchant, any size" true rather than aspirational.

DOOR 1 No website — a QR code is the storefront

the hawker uncle

They get a hosted page at `foodflow.sg/a/sakura` the moment they publish. We print the QR; they tape it to the stall front. A customer scans it and the agent opens full-screen on their phone. No website was ever involved.

WHAT THE MERCHANT DOES

Photographs their menu. Checks the tree. Taps **Publish**. That is the entire integration.

DOOR 2 Has a site builder — an app, never a snippet

the small café

A `<script>` tag is already too much for this person. Ship a FoodFlow app in the Wix, Squarespace, Shopify and WordPress marketplaces, so it is **Install** → **pick your stall** → **Done**. The snippet still exists underneath for anyone who wants it.

WHAT THE MERCHANT DOES

Clicks Install in a store they already use.
Never sees code.

DOOR 3 Already on an aggregator — import, don't upload

anyone on GrabFood

The best menu file is the one they already keep current. Pull the menu from their delivery-platform listing and enrich that. Onboarding becomes "paste your GrabFood link", and their catalog starts accurate instead of two years stale.

WHAT THE MERCHANT DOES

Pastes one link. Verifies the tree.

DOOR 4 Has a POS — live prices and real stock

the chain

API and webhooks. Prices and sold-out flags stay live, and confirmed orders push straight into the kitchen queue. This is the only door with an engineer behind it, and it is the door that makes the product credible at scale.

WHAT THE MERCHANT DOES

Their IT person spends an afternoon.
Everyone else spends none.

Your analogy is right but it has a trap in it. A developer console assumes a developer. Ours has to be usable by someone whose entire software experience is WhatsApp and a phone camera — so the design constraint is severe, and it points somewhere specific.

You don't fill in a form to get an agent. You talk to one.

Onboarding is itself a conversation. The merchant messages a number — **on WhatsApp, because that is what hawkers in Singapore actually use** — and sends a photo of the menu board. The onboarding agent reads it, asks the two or three things it genuinely can't infer ("is the chicken rice halal?", "how long for the katsu?"), and sends back a link and a QR code.

That is the bridge you're asking about. Not an SDK, not a dashboard tour: **the same conversational interface we're selling to customers, pointed at the merchant.** It's the strongest line in the pitch because it proves the thesis on the merchant side too — and it costs almost nothing to demo, because you already have the widget.

The console still exists — for editing prices, toggling sold-out, and seeing orders. But it is the *second* thing a merchant touches, not the first. Every screen in it answers one question: what do I do next. No JSON, no API keys on the front page, no word like "endpoint" anywhere a merchant can see.

Getting paid	The hard part of any payments product isn't taking money, it's paying the merchant — KYC, a merchant account, settlement. AI doesn't solve it. Our answer: the stall already accepts card or PayNow, so FoodFlow settles into the account they already have, and onboarding reuses the acquirer's existing KYC. Mock it in the demo, but say this out loud — a Visa judge will respect that you know where the real friction is.
Menu drift	Prices change and things sell out — our own NUS data is already stale, which proves the point. The console's quick-edit is the floor, Door 3's aggregator sync is better, Door 4's POS feed is the real fix. Turn the weakness into the argument for the product.
Who pays for the AI	Every conversation costs money and a hawker has no subscription budget. Charge a take rate on completed transactions , not a monthly fee — the merchant pays only when they've been paid.
Wrong allergen data	An agent that confidently says "no shellfish" and is wrong is a real-world harm, not a bug. Hence: allergens are a hard filter the merchant verifies , never a soft score, never inferred silently, and low-confidence rows are flagged rather than guessed. Say this before they ask it.

THE 30-SECOND ANSWER, IF A JUDGE ASKS HOW MERCHANTS ADOPT THIS

"Most of our merchants don't have a website, so we don't ask them to install anything. They send us a photo of their menu — over WhatsApp — and our agent turns it into a structured catalog with flavour, texture and allergen data. They check it and tap publish."

"That gives them a QR code for the stall, a one-click app if they have a Wix site, or an API if they have a POS. All four are the same agent underneath. The merchant never sees code, and the first thing they ever talk to is the product itself."

Companion to the FoodFlow prototype · the tree in Part 1 is the live output shape of the merchant console's catalog agent